SPI FULL-DUPLEX

Name : Gautam Raj Chhabra
Project : SPI Full-Duplex
Language : SystemVerilog
Tool Used : Xilinx Vivado

1. Abstract

This project implements a full-duplex SPI (Serial Peripheral Interface) using SystemVerilog. The design includes both Master and Slave modules with configurable CPOL/CPHA, supporting configurable CPOL/CPHA (verified for 2 SPI modes). The system enables simultaneous data transmission and reception. Verification is performed using a SystemVerilog testbench, assertions, and waveform analysis to ensure correct protocol behaviour and timing accuracy.

2. Introduction

SPI is a synchronous serial communication protocol used for high-speed data transfer between a master and peripheral devices.
This project includes:

- SPI Master
- SPI Slave
- Configurable clock polarity (CPOL) and phase (CPHA) *(verified for 2 SPI modes)*
- Full-duplex communication
- SystemVerilog-based verification

The master generates the clock and controls communication, while the slave responds accordingly.

3. Operational Conditions

- **IDLE**: Slave Select (SS) is high, no communication
- **TRANSFER**: SS goes low, enabling communication; data is transmitted on MOSI and received on MISO based on CPOL/CPHA configuration
- **STOP**: SS goes high, terminating communication
- **CLOCK (SCLK)**: Generated by master during transfer
- **FULL-DUPLEX**: Simultaneous transmit and receive

4. Design Features

- Full-duplex SPI communication
- Configurable CPOL/CPHA (supports multiple SPI modes)
- Master-Slave architecture
- Shift-register-based data transfer
- FSM-based control logic
- Assertion-based verification
- Randomized testbench
- Debugged clock alignment and sampling issues

5. Design Code

The SPI design is implemented using modular SystemVerilog components including master, slave, and interface.

- The **master** generates SCLK and controls SS
- Data is transmitted using **shift registers**
- The **slave** samples data based on clock edges
- FSM controls transmission states (IDLE, TRANSFER, DONE)

Proper handling of clock edges ensures correct synchronization between master and slave.

```
interface spi_if;

  logic clk;
  logic reset;
  logic start;

  logic mode;

  logic sclk;
  logic ss;
  logic mosi;
  logic miso;
  logic done;

  logic [7:0] master_tx;
  logic [7:0] master_rx;

  logic [7:0] slave_tx;
  logic [7:0] slave_rx;
```

```
  modport spi_master_mp (
    input  clk, reset, start, miso, master_tx, mode,
    output sclk, ss, mosi, done, master_rx
  );

  modport spi_slave_mp (
    input  sclk, ss, mosi, slave_tx, mode,
    output miso, slave_rx
  );

endinterface


module spi_master(spi_if.spi_master_mp intf);

  typedef enum logic [1:0] {IDLE, TRANSFER, DONE} state_t;
  state_t state, next_state;

  logic [2:0] bit_count;
  logic [7:0] shift_tx, shift_rx;
  logic sclk_en;

  logic cpol, cpha;

  assign cpol = intf.mode;
  assign cpha = intf.mode;

  always_ff @(posedge intf.clk or posedge intf.reset)
begin
    if (intf.reset)
      state <= IDLE;
    else
      state <= next_state;
  end

  always_comb begin
    case (state)
      IDLE:     next_state = intf.start ? TRANSFER : IDLE;
```

```
      TRANSFER: next_state = (bit_count == 3'd7 && sclk_en
&& (intf.sclk == ~cpol)) ? DONE : TRANSFER;
      DONE:     next_state = IDLE;
      default:  next_state = IDLE;
    endcase
  end

  always_ff @(posedge intf.clk or posedge intf.reset)
begin
    if (intf.reset) begin
      intf.sclk <= 0;
      intf.ss <= 1;
      intf.mosi <= 0;
      intf.done <= 0;
      intf.master_rx <= 0;
      bit_count <= 0;
      shift_tx <= 0;
      shift_rx <= 0;
      sclk_en <= 0;
    end
    else begin
      case (state)

        IDLE: begin
          intf.ss <= 1;
          intf.sclk <= cpol;
          intf.done <= 0;
          bit_count <= 0;
          sclk_en <= 0;

          if (intf.start) begin
            intf.ss <= 0;
            shift_tx <= intf.master_tx;
            shift_rx <= 0;
            sclk_en <= 1;

            if (cpha == 0)
              intf.mosi <= intf.master_tx[7];
          end
```

```
        end

        TRANSFER: begin
          if (sclk_en) begin
            intf.sclk <= ~intf.sclk;

            // MODE HANDLING
            if (intf.sclk == cpol) begin
              // SHIFT edge
              intf.mosi <= shift_tx[7];
              shift_tx <= {shift_tx[6:0], 1'b0};
            end
            else begin
              // SAMPLE edge
              shift_rx <= {shift_rx[6:0], intf.miso};
              bit_count <= bit_count + 1;
            end
          end
        end

        DONE: begin
          intf.ss <= 1;
          intf.sclk <= cpol;
          intf.done <= 1;
          intf.master_rx <= shift_rx;
          sclk_en <= 0;
        end

      endcase
    end
  end



property p_ss_active;
  @(posedge intf.clk)
  (state == TRANSFER) |-> (intf.ss == 0);
endproperty
```

```
assert property (p_ss_active)
  else $error("SPI ERROR: SS not LOW during TRANSFER");



property p_8bit_transfer;
  @(posedge intf.clk)
  (state == TRANSFER && next_state == DONE) |-> (bit_count
== 3'd7);
endproperty

assert property (p_8bit_transfer)
  else $error("SPI ERROR: Transfer not exactly 8 bits");


property p_idle_sclk;
  @(posedge intf.clk)
  (state == IDLE) |-> (intf.sclk == cpol);
endproperty

assert property (p_idle_sclk)
  else $error("SPI ERROR: SCLK not matching CPOL in
IDLE");



endmodule



module spi_slave(spi_if.spi_slave_mp intf);

  logic [2:0] bit_count;
  logic [7:0] shift_tx, shift_rx;

  logic cpol, cpha;
  assign cpol = intf.mode;
  assign cpha = intf.mode;
```

```
  always_ff @(negedge intf.ss) begin
    shift_tx <= intf.slave_tx;
    shift_rx <= 0;
    bit_count <= 0;

    if (cpha == 0)
      intf.miso <= intf.slave_tx[7]; // Mode0 preload
  end


  always_ff @(posedge intf.sclk or negedge intf.sclk)
begin
    if (!intf.ss) begin
      if ((intf.sclk == ~cpol)) begin
        shift_rx <= {shift_rx[6:0], intf.mosi};
        bit_count <= bit_count + 1;
      end
    end
  end

  always_ff @(posedge intf.sclk or negedge intf.sclk)
begin
    if (!intf.ss) begin
      if ((intf.sclk == cpol)) begin
        shift_tx <= {shift_tx[6:0], 1'b0};
        intf.miso <= shift_tx[6];
      end
    end
  end

    always_ff @(posedge intf.ss) begin
    intf.slave_rx <= shift_rx;
  end

endmodule
```

6. Testbench

The testbench is developed using SystemVerilog with randomized stimulus.

- Interface-based connection between modules
- Random data generation
- Master-Slave communication verification
- Assertions to check protocol correctness
- Waveform analysis for debugging

```
`timescale 1ns/1ps

module tb;

  spi_if intf();

  spi_master master (intf);
  spi_slave slave (intf);

  always #5 intf.clk = ~intf.clk;

  class spi_trans;
    rand bit mode;
    rand bit [7:0] master_tx;
    rand bit [7:0] slave_tx;
  endclass

  task run_transfer(spi_trans tr);
    begin
      intf.mode      = tr.mode;
      intf.master_tx = tr.master_tx;
      intf.slave_tx  = tr.slave_tx;

      #20;
      intf.start = 1;
      #10;
      intf.start = 0;

      wait(intf.done);

      #40;
    end
  endtask
```

```
  initial begin
    spi_trans tr1, tr2;

    intf.clk = 0;
    intf.reset = 1;
    intf.start = 0;
    intf.mode  = 0;

    #20;
    intf.reset = 0;


    tr1 = new();
  tr1.randomize() with { mode == 0; };
    run_transfer(tr1);


    tr2 = new();
    tr2.randomize() with { mode == 1; };
    run_transfer(tr2);

    #10000;
    $finish;
  end

  initial begin
    $monitor("TIME=%0t | MODE=%0d | SS=%b SCLK=%b MOSI=%b
MISO=%b | M_TX=%h S_TX=%h | M_RX=%h S_RX=%h DONE=%b",
             $time, intf.mode,
             intf.ss, intf.sclk, intf.mosi, intf.miso,
             intf.master_tx, intf.slave_tx,
             intf.master_rx, intf.slave_rx,
             intf.done);
  end

endmodule
```

7. Simulation Result

Simulation verifies correct SPI communication:

- Data transmitted by master is correctly received by slave
- Data received by master matches slave transmission
- No data corruption observed

```
TIME=135000 | MODE=0 | SS=0 SCLK=1 MOSI=1 MISO=1 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=145000 | MODE=0 | SS=0 SCLK=0 MOSI=1 MISO=0 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=155000 | MODE=0 | SS=0 SCLK=1 MOSI=0 MISO=0 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=165000 | MODE=0 | SS=0 SCLK=0 MOSI=0 MISO=1 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=175000 | MODE=0 | SS=0 SCLK=1 MOSI=0 MISO=1 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=185000 | MODE=0 | SS=0 SCLK=0 MOSI=0 MISO=1 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=195000 | MODE=0 | SS=0 SCLK=1 MOSI=1 MISO=1 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=205000 | MODE=0 | SS=0 SCLK=0 MOSI=1 MISO=0 | M_TX=39 S_TX=bb | M_RX=00 S_RX=xx DONE=0
TIME=215000 | MODE=0 | SS=1 SCLK=0 MOSI=1 MISO=0 | M_TX=39 S_TX=bb | M_RX=bb S_RX=39 DONE=1
TIME=225000 | MODE=0 | SS=1 SCLK=0 MOSI=1 MISO=0 | M_TX=39 S_TX=bb | M_RX=bb S_RX=39 DONE=0
TIME=255000 | MODE=1 | SS=1 SCLK=1 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=275000 | MODE=1 | SS=0 SCLK=1 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=285000 | MODE=1 | SS=0 SCLK=0 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=295000 | MODE=1 | SS=0 SCLK=1 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=305000 | MODE=1 | SS=0 SCLK=0 MOSI=0 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=315000 | MODE=1 | SS=0 SCLK=1 MOSI=0 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=325000 | MODE=1 | SS=0 SCLK=0 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=335000 | MODE=1 | SS=0 SCLK=1 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=345000 | MODE=1 | SS=0 SCLK=0 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=355000 | MODE=1 | SS=0 SCLK=1 MOSI=1 MISO=1 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=365000 | MODE=1 | SS=0 SCLK=0 MOSI=0 MISO=1 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=375000 | MODE=1 | SS=0 SCLK=1 MOSI=0 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=385000 | MODE=1 | SS=0 SCLK=0 MOSI=0 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=395000 | MODE=1 | SS=0 SCLK=1 MOSI=0 MISO=1 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=405000 | MODE=1 | SS=0 SCLK=0 MOSI=0 MISO=1 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=415000 | MODE=1 | SS=0 SCLK=1 MOSI=0 MISO=1 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=425000 | MODE=1 | SS=0 SCLK=0 MOSI=1 MISO=1 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=435000 | MODE=1 | SS=0 SCLK=1 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=bb S_RX=39 DONE=0
TIME=445000 | MODE=1 | SS=1 SCLK=1 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=0b S_RX=b1 DONE=1
TIME=455000 | MODE=1 | SS=1 SCLK=1 MOSI=1 MISO=0 | M_TX=b1 S_TX=0b | M_RX=0b S_RX=b1 DONE=0
$finish called at time : 535 ns : File "C:/vivado/vending/sp2/sp2.srcs/sim_1/new/sp2_tb.sv" Line 66
```

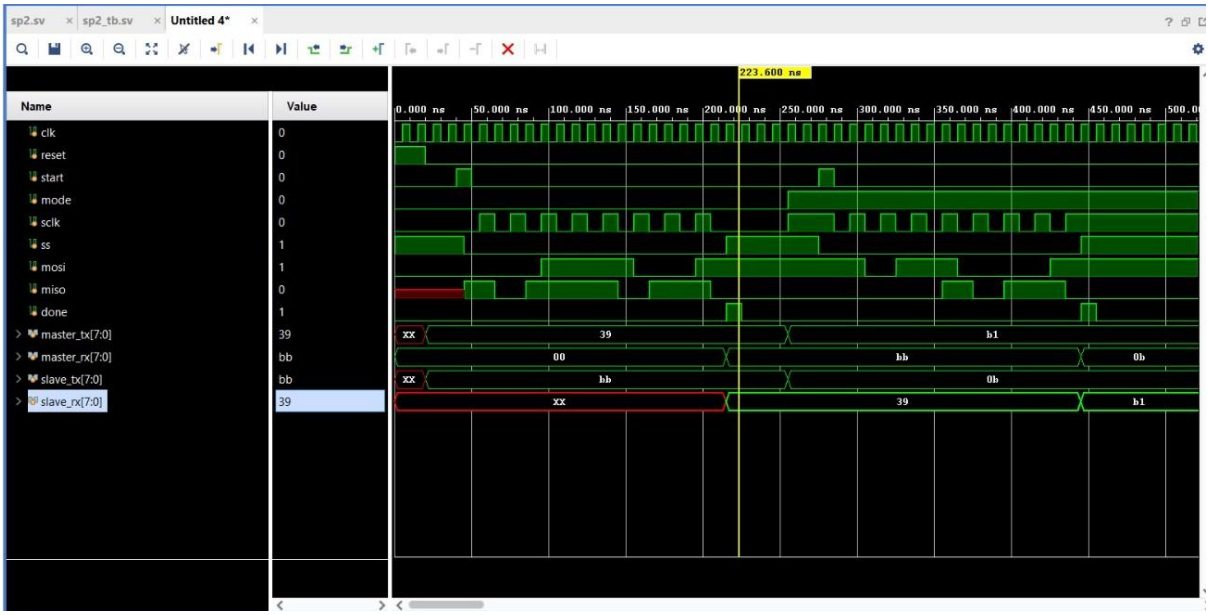
8. Waveform

The waveform demonstrates correct SPI behavior:

- SS goes low to start communication
- SCLK toggles based on CPOL
- Data is sampled based on CPHA
- Simultaneous transmit and receive observed

Observations:

- Proper clock-data alignment
- No glitches in communication
- Stable full-duplex transfer

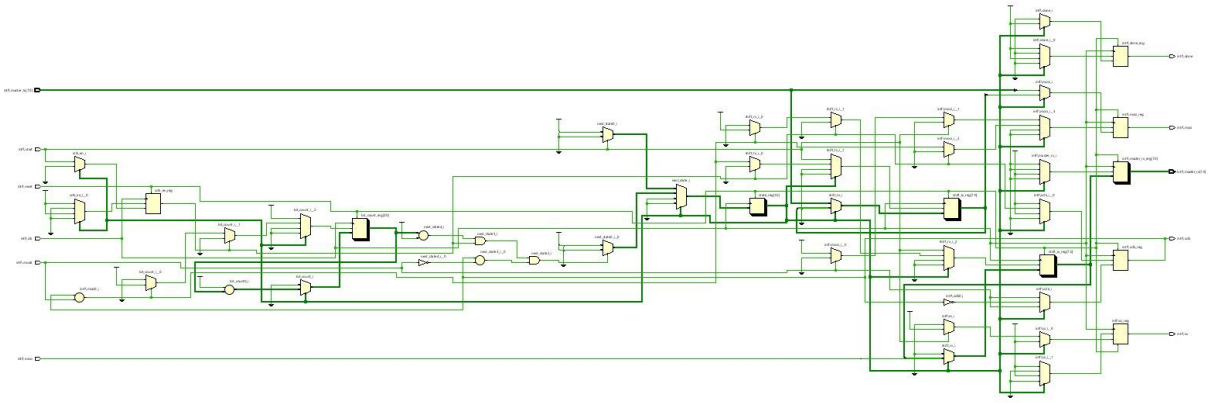


9. RTL Schematic

The RTL schematic includes:

- FSM for master and slave
- Shift registers for data transfer
- Clock generation logic
- Control signals (SS, SCLK, MOSI, MISO)

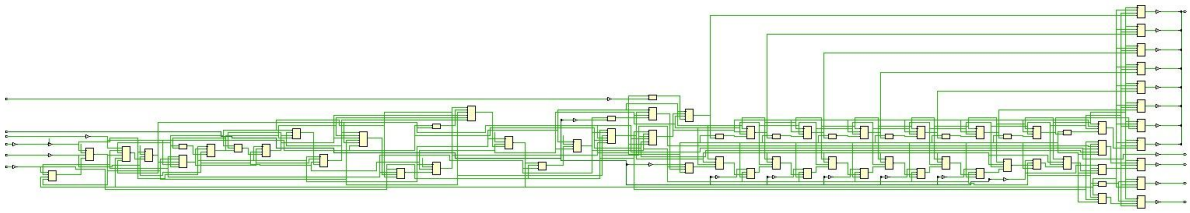
This confirms correct structural design.



10. Synthesis Schematic

The synthesized design verifies hardware implementation:

- No unintended latches
- Proper mapping of FSM and registers
- Clean timing paths
- Fully synthesizable design



11. Challenges & Fixes

Problems faced:

- Data mismatch between master and slave
- Incorrect sampling edge
- Clock alignment issues
- Timing mismatch in full-duplex transfer

Solutions:

1. Correct CPOL/CPHA handling
2. Fixed sampling edge alignment
3. Improved shift register timing
4. Verified correct data capture points

12. Conclusion

The SPI system is successfully designed and verified.

- Achieved reliable full-duplex communication
- Implemented configurable SPI modes
- Debugged real timing issues
- Verified using assertions and simulation

This project demonstrates strong understanding of synchronous communication and timing-aware digital design.